Institut Supérieur d'Informatique et de Gestion (ISIGK)

Department of Computer Science

Robotics Club Management Platform

Technical Project Report

Project Supervisors :	Pr. [Name] & [Name]
Club Leader :	[Name]
Co-Leader :	[Name]
Development Team	[Names]
Academic Year :	2026 – 2027

October 15, 2026

Contents

1 General Introduction **4**
- 1.1 Project Overview . . . 4
- 1.2 Problem Statement . . . 4
- 1.3 Project Objectives . . . 5

2 Project Context **6**
- 2.1 Project Scope and Constraints . . . 6
  - 2.1.1 Scope . . . 6
  - 2.1.2 Constraints . . . 6
  - 2.1.3 Assumptions . . . 7
- 2.2 Chapter Conclusion . . . 7

3 State of the Art **8**
- 3.1 Existing solutions . . . 8
  - 3.1.1 General messaging tools . . . 8
  - 3.1.2 Collaboration and productivity suites . . . 8
  - 3.1.3 Project and task management . . . 8
  - 3.1.4 Club / association management software . . . 8
  - 3.1.5 Communication infrastructure technologies . . . 8
- 3.2 Synthesis and Positioning of the project . . . 9
- 3.3 Wrap-up . . . 9

4 Requirements & Planning **10**
- 4.1 Methodology . . . 10
- 4.2 Stakeholder Identification . . . 11
- 4.3 Functional Requirements . . . 11
- 4.4 Non Functional Requirements . . . 12
- 4.5 Product Backlog . . . 13
  - 4.5.1 Epic summary . . . 13
  - 4.5.2 User stories — MVP . . . 13
  - 4.5.3 User stories — Video experiment (TEST) . . . 15
  - 4.5.4 User stories — Production version (PROD) . . . 15
- 4.6 Sprint / Chapter Planning . . . 15
- 4.7 Conclusion . . . 16

5 Sprint / Chapter 0 **17**
- 5.1 Introduction . . . 17
- 5.2 Actors . . . 17
- 5.3 Diagrams . . . 19

5.4 Technology Stack 19
5.5 Review & Retrospective 19
5.6 Conclusion 19

6 Sprint / Chapter 1 **20**
6.1 Introduction 20
6.2 Actors 20
6.3 Diagrams 20
6.4 Technology Stack 20
6.5 Review & Retrospective 20
6.6 Conclusion 20

7 General Conclusion **21**

Bibliography **22**

A Appendix A — Database draft **23**

B Appendix B — Screenshots **24**

C Appendix C — Meeting minutes **25**

D Appendix D — Backlog export **26**

Webography **27**

List of Tables

List of Figures

5.1 Global use case diagram — to be delivered in Sprint 0 18
5.2 Data model — to be delivered in Sprint 0 18
5.3 Technical architecture — to be delivered in Sprint 0 18

Chapter 1

General Introduction

1.1 Project Overview

The Robotics Club of ISIGK is a student association whose mission is to promote robotics, electronics and innovation among students through workshops, competitions and collaborative projects. The club is governed by a well-defined hierarchical organization: one professor acting as an external supervisor, one leader and one co-leader directing the strategy, and five managers each in charge of a specific department (Human Resources, Events, Media, Finance and Materials). Each department contains members, and the club also welcomes guest members who are in an observation or trial period.

This project aims to design and develop a **web-based management platform** that digitizes the club's daily operations. The platform provides each actor with a dedicated interface adapted to his role and responsibilities: member registry management, event organization, internal communication through chat, budget and inventory tracking, media publication, and real-time notifications.

The deliverable of this project follows an **agile, sprint-based approach**: each sprint is documented as a chapter of this report, allowing continuous tracking of the progress from the initial backlog to the final production-ready product.

1.2 Problem Statement

Despite its dynamic organization, the Robotics Club currently suffers from a fragmented and informal management style:

- **Scattered communication:** exchanges take place in multiple WhatsApp and Telegram groups, mixing administrative, technical and social content. Important information gets lost and guests or new members are often excluded.
- **No structured information system:** member lists, attendance and department data are stored in personal files (Excel, Word) with no single source of truth and no access control.
- **Manual event organization:** invitations, RSVP tracking and post-event follow-up are done manually, leading to duplicated effort and low participation visibility.
- **Untracked resources:** materials, electronic components and tools are borrowed without traceability, causing losses and stock discrepancies.

- **Opaque finances:** income, expenses and per-department budgets are not visible to the club's supervision, limiting accountability.
- **No remote collaboration:** there is no integrated solution for video meetings tied to the club's departments and events.

Existing general-purpose tools (WhatsApp, Google Drive, Discord, Notion) each cover only a fragment of these needs and do not enforce the club's specific role hierarchy. There is therefore a real need for a **custom platform aligned with the club's organization**.

1.3 Project Objectives

The main objective is to build a single platform that makes it easy for every user to manage his role within the club. The specific objectives are:

1. **O1 – Role-based access:** implement a strict permission model (Professor, Leader, Co-leader, Manager, Member, Guest) so that each user sees and does exactly what his role allows.
2. **O2 – Department management:** provide the five managers with the tools to run their departments (HR, Events, Media, Finance, Materials).
3. **O3 – Unified communication:** integrate internal chat (direct messages and channels) with file and image sharing.
4. **O4 – Collaboration:** enable video meetings, tested first with an open-source solution before the final choice.
5. **O5 – Real-time awareness:** deliver in-app notifications for messages, events, tasks and alerts.
6. **O6 – Guest integration:** offer a smooth pipeline from public visitor to guest, then to confirmed club member.
7. **O7 – Traceability:** keep history of activities, finances and material loans for accountability towards the supervisor.
8. **O8 – Progressive delivery:** ship a Minimum Viable Product (MVP) first, then iterate towards a production-grade version (PROD).

Chapter 2

Project Context

2.1 Project Scope and Constraints

2.1.1 Scope

The platform covers the following functional perimeter:

In scope	Out of scope (v1)
Authentication	Payment gateway integration
Role management (6 roles)	Native mobile application
Departments & members registry	Email/Telegram notifications (PROD)
Guest application pipeline	Advanced analytics / BI
Events & RSVP	Self-hosted video infrastructure
Tasks assignment	Full-text search engine
Chat (DM, channels, attachments)	Multi-language UI (PROD)
In-app notifications	
Finance & materials tracking	
Media gallery & announcements	
Public landing page	

2.1.2 Constraints

- **Human:** a small student development team, working alongside academic obligations; sprints must be realistic (1–2 weeks).

- **Technical:** video conferencing must not be self-hosted in v1 (bandwidth and maintenance costs); an external open-source service (Jitsi) is evaluated first.
- **Financial:** the budget is limited to free tiers and open-source tooling.
- **Organizational:** the platform must respect the club hierarchy; the professor has read-only access to everything, including chats.
- **Legal:** personal data of members must be handled responsibly (password hashing, restricted access).

2.1.3 Assumptions

- The club structure (1 professor, 1 leader, 1 co-leader, 5 managers, members, guests) is stable for the duration of the project.
- The user base remains in the order of tens of users, not thousands.
- Users access the platform mainly from laptops and smartphones via a web browser.

2.2 Chapter Conclusion

The project is bounded by tight human and financial constraints but targets a clearly identified pain: the absence of a unified, role-aware information system for the club. By restricting the scope to a web application with proven open-source technologies, and by deferring heavy features (self-hosted video, email alerts) to the production phase, the project remains achievable by a student team while delivering real value in the MVP.

Chapter 3

State of the Art

3.1 Existing solutions

Several categories of existing solutions were analyzed:

3.1.1 General messaging tools

WhatsApp, **Telegram** and **Discord** [?] dominate student association communication. They provide groups, media sharing and voice channels, but offer no role hierarchy matching a club organization, no member registry, and no traceability of administrative data. Discord comes closest with text/voice channels and role tags, yet it remains a communication tool and not a management system.

3.1.2 Collaboration and productivity suites

Google Workspace (Drive, Sheets, Forms) and **Notion** allow clubs to store documents, build simple databases and forms [?]. They are flexible but require manual configuration, lack a coherent permission model, and provide no integrated chat, notifications or event workflow.

3.1.3 Project and task management

Trello, **Jira** and **Asana** [?] offer boards, tasks and deadlines. They fit development teams better than club organizations, and again provide no finance, inventory or membership features.

3.1.4 Club / association management software

Products such as **WildApricot**, **MembershipWorks** or management modules of **Moodle** [?] address membership, dues and events. They are designed for Western associations, are often paid per member, and do not natively support department hierarchies, chat or inventory of electronic components.

3.1.5 Communication infrastructure technologies

For real-time features, the state of the art converges on **WebSocket** technologies, commonly implemented with **Socket.io** on Node.js applications [?]. For video meetings, the

dominant open-source option is **Jitsi Meet** [?], which can be embedded in a web page through an iframe without operating any media server, at zero cost.

3.2 Synthesis and Positioning of the project

Solution	Roles	Chat	Events	Finance	Stock	Free
WhatsApp / Telegram	–		partial	–	–	
Discord	partial		–	–	–	
Google Drive/Sheets	–	–	partial	partial	partial	
Notion	partial	–	partial	partial	partial	partial
Trello / Jira	partial	–	–	–	–	partial
WildApricot / Moodle		–			–	–
Our platform						

No single existing solution combines: (i) the exact club hierarchy with a strict read-only supervisor, (ii) department-scoped management modules, (iii) chat and notifications, and (iv) zero operating cost. The project therefore positions itself as a **lightweight, open-source, role-aware club information system**, built with a student-friendly JavaScript stack and leveraging external free services (Jitsi) for heavy features.

3.3 Wrap-up

The state of the art confirms the feasibility of every targeted feature: real-time chat with Socket.io, embedded video with Jitsi, and role-based web applications with component-based front-end frameworks. The differentiation of this project lies in its **integration** and in its **permission model**, not in technological novelty. The next chapter formalizes these needs into requirements, a product backlog and a sprint plan.

Chapter 4

Requirements & Planning

4.1 Methodology

The project follows **Scrum** [?] with short sprints:

- **Product Owner:** the club Leader (validates the backlog and priorities).
- **Scrum Master:** the development team lead (facilitates the process).
- **Development Team:** the student developers.
- **Sprint duration:** 1 to 2 weeks; each sprint is reported as one "Sprint/Chapter" of this document.
- **Artifacts:** product backlog (Section 4.5), sprint backlog, increment (working software).
- **Ceremonies:** sprint planning, daily sync, sprint review with the club leaders, sprint retrospective (Section 5.5 of each chapter).
- **Definition of Done:** feature coded, tested manually, documented and demoed to the Product Owner.

Priorities use the MoSCoW-like notation: **MVP** (must have in the MVP), **TEST** (evaluated experimentally), **PROD** (deferred to the production version).

4.2 Stakeholder Identification

Stakeholder	Role w.r.t. project	Involvement
Professor	Supervisor / end user (read-only)	Validates that the club activities are traceable and visible
Club Leader	Product Owner	Prioritizes the backlog, validates increments
Co-Leader	Deputy Product Owner	Covers the Leader's validations
HR Manager	End user	Manages members, attendance, applications
Events Manager	End user	Manages events, RSVPs, reminders
Media Manager	End user	Manages the gallery and publications
Finance Manager	End user	Manages income, expenses, budgets
Materials Manager	End user	Manages inventory and loans
Members	End users	Use tasks, chats, events
Guests	External users	Public site, application form
Development Team	Builders	Designs, develops, tests and deploys the platform
ISIGK	Host institution	Provides identity and context

4.3 Functional Requirements

ID	Requirement	Priority	Sprint
FR-01	Register / login with email and password	MVP	S1
FR-02	Assign, promote and demote roles (guest → member → manager)	MVP	S1
FR-03	Professor account with global *read-only* access (data + chats)	MVP	S1
FR-04	Manage the 5 departments and the Board team	MVP	S1
FR-05	Members directory with search and department filter	MVP	S2
FR-06	User profile (name, role, department, photo, bio)	MVP	S2
FR-07	Guest application form and approval pipeline	MVP	S2
FR-08	Create / edit / delete events	MVP	S3
FR-09	RSVP and attendee list per event	MVP	S3
FR-10	Assign tasks with deadline and status	MVP	S3
FR-11	Direct messages between users	MVP	S4
FR-12	Department channels (auto-joined) and club-wide channel	MVP	S4

ID	Requirement	Priority	Sprint
FR-13	Private Board channel (Leader, Co-leader, Professor)	MVP	S4
FR-14	File and image attachments inside chat	MVP	S4
FR-15	Real-time in-app notification center	MVP	S3–S4
FR-16	Record income and expenses with proof	MVP	S5
FR-17	Per-department budgets and monthly report	MVP	S5
FR-18	Materials inventory with quantities and states	MVP	S5
FR-19	Loan / reservation of materials with return date	MVP	S5
FR-20	Low-stock automatic alert	MVP	S5
FR-21	Media gallery (photos, videos)	MVP	S6
FR-22	Public landing page and join form	MVP	S6
FR-23	Announcements from Leader broadcast to all	MVP	S6
FR-24	Role-adapted dashboards (KPIs)	MVP	S2–S6
FR-25	Video meeting button per channel / event	TEST	S4 (test)
FR-26	Email notifications	PROD	PROD
FR-27	Mentions and read receipts	PROD	PROD
FR-28	Kanban task board	PROD	PROD
FR-29	Member dues tracking	PROD	PROD
FR-30	Purchase requests (Materials → Finance)	PROD	PROD
FR-31	Extra sub-channels created by managers	PROD	PROD
FR-32	Post-event reports with photos	PROD	PROD
FR-33	Social media posts planner	PROD	PROD
FR-34	Multi-language interface (FR / EN)	PROD	PROD
FR-35	Notification preferences per user	PROD	PROD

4.4 Non Functional Requirements

Category	Requirement
Security	Passwords hashed (bcrypt), JWT sessions, strict server-side role checks, protection against SQL injection and XSS
Confidentiality	A user only accesses his department's data; the Leader, Co-leader and Professor access everything; the Professor cannot modify anything
Performance	Pages load in $<$ 2s on a standard campus connection; chat messages delivered in $<$ 500ms
Availability	Hosted on a free-tier service with $\geq 95\%$ availability; graceful degradation if the video service is down
Usability	Responsive interface (desktop + mobile), French/English friendly, discoverable navigation per role
Reliability	Daily database backups; no data loss on server restart
Scalability	Architecture able to serve 100+ users without redesign
Maintainability	Modular code (one module per department), documented API, version control with Git
Portability	Runs on any system with a modern browser; deployable on Linux hosting

4.5 Product Backlog

4.5.1 Epic summary

Epic	Title	Contains
E1	Authentication & Roles	Login, roles, professor read-only
E2	Departments & Members	Registry, directory, profiles, guest pipeline
E3	Events	Events, RSVP, reminders
E4	Tasks	Assignment, deadlines, statuses
E5	Chat	DM, channels, attachments
E6	Video Meetings	Jitsi experiment, meeting buttons
E7	Notifications	In-app center, real-time
E8	Finance	Income, expenses, budgets, reports
E9	Materials	Inventory, loans, stock alerts
E10	Media	Gallery, public preview
E11	Announcements	Leader broadcasts
E12	Public Site	Landing page, join form
E13	Settings	Profile, password, preferences
E14	Dashboards	Role-adapted KPIs

4.5.2 User stories — MVP

ID	As a . . . I want to . . . so that . . .	Epic
US-1.1	As a **visitor** I want to register and log in so that I access the platform	E1
US-1.2	As a **leader** I want to assign roles so that each user gets the right permissions	E1
US-1.3	As a **professor** I want to view all data and chats read-only so that I supervise the club	E1
US-2.1	As a **leader** I want the 5 departments created so that the club is structured	E2
US-2.2	As a **manager** I want to see the members of my department so that I coordinate them	E2
US-2.3	As a **member** I want a profile so that others identify me	E2
US-2.4	As a **guest** I want to apply to join a department so that I become a member	E2
US-2.5	As a **manager** I want to accept or refuse applications so that I control my team	E2
US-3.1	As an **events manager** I want to create events so that the club is informed	E3
US-3.2	As a **member** I want to RSVP so that organizers plan properly	E3
US-3.3	As a **member** I want to receive event reminders so that I don't miss them	E3

ID	Story	Epic
US-4.1	As a **manager** I want to assign tasks to my members so that work progresses	E4
US-4.2	As a **leader** I want to assign tasks to managers so that objectives are tracked	E4
US-4.3	As a **member** I want to update my task status so that everyone sees progress	E4
US-5.1	As a **member** I want to send direct messages so that I communicate privately	E5
US-5.2	As a **member** I want a department channel so that my team discusses in one place	E5
US-5.3	As a **member** I want a club-wide channel so that news reaches everyone	E5
US-5.4	As a **leader** I want a private Board channel so that strategy stays confidential	E5
US-5.5	As a **member** I want to share files and images in chat so that we exchange documents	E5
US-7.1	As a **user** I want a notification center so that I never miss anything	E7
US-7.2	As a **user** I want real-time notifications so that I don't refresh the page	E7
US-8.1	As a **finance manager** I want to record income and expenses so that finances are traceable	E8
US-8.2	As a **leader** I want per-department budgets so that spending is controlled	E8
US-8.3	As a **professor** I want a monthly financial report so that I supervise the club budget	E8
US-9.1	As a **materials manager** I want an inventory so that stock is known	E9
US-9.2	As a **member** I want to borrow material with a return date so that items come back	E9
US-9.3	As a **materials manager** I want low-stock alerts so that I reorder in time	E9
US-10.1	As a **media manager** I want to upload photos and videos so that activities are showcased	E10
US-10.2	As a **visitor** I want to see a public gallery preview so that I discover the club	E10
US-11.1	As a **leader** I want to post announcements so that the whole club is informed	E11
US-12.1	As a **visitor** I want a landing page so that I understand the club	E12
US-12.2	As a **visitor** I want a join form so that I become a guest	E12
US-13.1	As a **user** I want to edit my profile and password so that my data stays correct	E13
US-14.1	As a **leader** I want club KPIs so that I pilot the club	E14

ID	Story	Epic
US-14.2	As a **manager** I want department KPIs so that I pilot my team	E14
US-14.3	As a **member** I want my tasks and my department news in one place	E14

4.5.3 User stories — Video experiment (TEST)

ID	Story	Epic
US-6.1	As a **user** I want a "Start meeting" button in a channel so that we talk face to face	E6
US-6.2	As a **manager** I want a permanent room for my department so that meetings are easy	E6
US-6.3	As a **member** I want a "Join meeting" button on events so that I attend remotely	E6

Decision gate: Jitsi Meet is prototyped in Sprint 4; the team accepts or rejects it in the Sprint 4 review before integrating it into the MVP.

4.5.4 User stories — Production version (PROD)

ID	Story	Epic
US-6.4	As a **user** I want meetings integrated natively	E6
US-7.3	As a **user** I want email notifications for important events	E7
US-7.4	As a **user** I want Telegram alerts so that I stay informed on my phone	E7
US-5.6	As a **user** I want @mentions and read receipts	E5
US-5.7	As a **manager** I want to create sub-channels in my department	E5
US-4.4	As a **manager** I want a kanban board so that I visualize tasks	E4
US-8.4	As a **finance manager** I want to track member dues	E8
US-9.4	As a **materials manager** I want purchase requests sent to Finance	E9
US-3.4	As an **organizer** I want a post-event report with photos	E3
US-10.3	As a **media manager** I want a social posts planner	E10
US-12.3	As a **visitor** I want the site in French and English	E12
US-13.2	As a **user** I want notification preferences	E13

4.6 Sprint / Chapter Planning

Each sprint is delivered and documented as one chapter of this report.

Sprint	Goal	Main deliverables	Chapter
S0	Project setup	Repo, environment, skeleton app, this backlog	Ch. 5
S1	Access control	Auth, 6 roles, RBAC, profiles	Ch. 6
S2	Members	Departments, directory, guest pipeline, dashboard base	Ch. 7
S3	Organization	Events, RSVP, tasks, notifications core	Ch. 8
S4	Communication	Chat (DM, channels, attachments) + video test (Jitsi)	Ch. 9
S5	Resources	Finance, budgets, inventory, loans, stock alerts	Ch. 10
S6	Visibility	Media gallery, announcements, public landing page	Ch. 11
S7	Hardening	Testing, bug fixing, deployment, demo data, handover	Ch. 12
—	**PROD phase**	Email, Telegram, kanban, dues, sub-channels...	Ch. 13+

4.7 Conclusion

This chapter transformed the club's needs into a formal requirement set: 35 functional requirements, 9 categories of non-functional requirements, a backlog of 14 epics and more than 50 user stories split between MVP, experimental and production scopes, and a sprint plan aligned with the report structure. The project is now ready to enter Sprint 0, which initializes the technical environment and the application skeleton.

Chapter 5

Sprint / Chapter 0

5.1 Introduction

Sprint 0 is an initialization sprint: it does not deliver business features but sets up everything the following sprints depend on – repository, environment, application skeleton, common conventions and the product backlog validated with the Product Owner.

Sprint goal: "A running skeleton application with authentication stubs and a validated backlog."

5.2 Actors

The actors of the system are formalized as follows (they are refined in every sprint):

Actor	Description	Main interactions
Professor	Club supervisor, read-only on everything	View dashboards, data, chats
Leader	Club head, full control	Manage all, assign roles, validate
Co-Leader	Deputy head	Same powers as the Leader
HR Manager	Head of Human Resources	Members, attendance, applications
Events Manager	Head of Events	Events, RSVP, reminders
Media Manager	Head of Media	Gallery, publications
Finance Manager	Head of Finance	Income, expenses, budgets
Materials Manager	Head of Materials	Inventory, loans, alerts
Member	Department member	Tasks, chats, events, loans
Guest	Visitor / trial user	Public site, application, limited chat

Use Case Diagram (UML)

[TODO: Insert `usecase_global.png` after Sprint 0 review]
Actors: Professor, Leader, Co-leader, 5 Managers, Member, Guest

Figure 5.1: Global use case diagram — to be delivered in Sprint 0

Class / Data Model Diagram

[TODO: Insert entity-relationship diagram (users, departments, events, messages, notifications, expenses, materials, . . .)]

Figure 5.2: Data model — to be delivered in Sprint 0

Architecture Diagram (3 tiers)

[TODO: Browser (React) → API (Node/Express) → Database; Socket.io channel; Jitsi iframe]

Figure 5.3: Technical architecture — to be delivered in Sprint 0

5.3 Diagrams

5.4 Technology Stack

To be defined and justified in the Sprint 0 review with the Product Owner. The shortlist discussed so far:

Layer	Candidates
Front-end	React + Vite, or Next.js
Back-end	Node.js + Express, or Next.js API routes
Database	SQLite (dev) → PostgreSQL (hosted)
Real-time	Socket.io
Video	Jitsi Meet (embedded) — **TEST**decision gate
Auth	JWT + bcrypt
Deployment	Free tier (Vercel / Railway / local server)

[TODO: Fill this table with the final choices and add a justification paragraph after the Sprint 0 review.]

5.5 Review & Retrospective

Sprint review: [TODO: to be filled after the review meeting with the Product Owner (date, demo feedback, backlog adjustments).]

What went well	What went wrong	Actions to improve
[TODO:]	[TODO:]	[TODO:]

5.6 Conclusion

Sprint 0 **[TODO: conclusion to be written after the sprint: environment status, backlog validation, decisions taken (stack, video solution), and outlook for Sprint 1.]**

Chapter 6

Sprint / Chapter 1

6.1 Introduction

[TODO: Sprint 1 goal: authentication, the 6 roles and role-based access control.]

6.2 Actors

[TODO: Actors concerned by this sprint and their permissions matrix.]

6.3 Diagrams

[TODO: Use case (auth), sequence diagram (login), class diagram (User, Role).]

6.4 Technology Stack

[TODO: Technologies used and difficulties encountered for this sprint.]

6.5 Review & Retrospective

[TODO: Review feedback and retrospective table.]

6.6 Conclusion

[TODO: Sprint 1 conclusion and outlook for Sprint 2.]

Chapter 7

General Conclusion

[TODO: To be written at the end of the project: recap of the initial problem, the delivered MVP, the PROD outlook, measured results (users, feedback from the club, metrics), individual learnings and perspectives (mobile app, native video, inter-club federation).]

Bibliography

[1] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. [Online]. Available: https://www.scrumguides.org/scrum-guide.html

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2019.

[3] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.

[4] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed. Prentice Hall, 2004.

[5] Mozilla Web Docs, "The WebSocket API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Appendix A

Appendix A — Database draft

[TODO: Physical data model, table descriptions and sample SQL schema.]

Appendix B

Appendix B — Screenshots

[TODO: Screenshots of each sprint increment.]

Appendix C

Appendix C — Meeting minutes

[TODO: Sprint reviews and Product Owner validation minutes.]

Appendix D

Appendix D — Backlog export

[TODO: CSV/board export of the product backlog.]

Webography

- Scrum Guide. https://www.scrumguides.org/
- React — official documentation. https://react.dev/
- Vite. https://vitejs.dev/
- Node.js. https://nodejs.org/
- Express.js. https://expressjs.com/
- Socket.io. https://socket.io/
- Jitsi Meet handbook. https://jitsi.github.io/handbook/
- SQLite. https://www.sqlite.org/
- PostgreSQL. https://www.postgresql.org/
- JSON Web Tokens. https://jwt.io/
- bcrypt. https://www.npmjs.com/package/bcrypt